

ASSIGNMENT 3

3.1.

3.1.1.

```
[3]: # Nguyễn Hoàng Tùng

import tensorflow as tf
h = tf.constant("Hello")
w = tf.constant(" World!")
hw = h + w
with tf.Session() as
    ans = sess.run(hw)
    print(ans)
```

Cell In[3], line 7
with tf.Session() as
 ^
SyntaxError: invalid syntax

```
[ ]:
```

This code has an invalid syntax, should be fixed as the image below.

```
[*]: # Nguyễn Hoàng Tùng

import tensorflow as tf

h = tf.constant("Hello")
w = tf.constant(" World!")
hw = h + w

with tf.Session() as sess:
    ans = sess.run(hw)
    print(ans)
```

C:\Python313\Lib\site-packages\google\protobuf\runtime_version.py:98: UserWarning: Protobuf gencode version 5.28.3 is exactly one major version older than the runtime version 6.31.1 at tensorflow/core/framework/attr_value.proto. Please update the gencode to avoid compatibility violations in the next runtime release.

```
-----
AttributeError                                Traceback (most recent call last)
Cell In[4], line 9
      6 w = tf.constant(" World!")
      7 hw = h + w
----> 9 with tf.Session() as sess:
     10     ans = sess.run(hw)
     11     print(ans)

AttributeError: module 'tensorflow' has no attribute 'Session'
```

This code has been fixed but can only run on Tensorflow 1.4. To run on version 2.0 we need to change the code as follows, no need for Session, just .numpy() to get the value.

```
[5]: # Nguyễn Hoàng Tùng

import tensorflow as tf

h = tf.constant("Hello")
w = tf.constant(" World!")
hw = h + w

print(hw.numpy().decode("utf-8"))
```

Hello World!

[]:

3.1.2.

```
[6]: # Nguyễn Hoàng Tùng

import tensorflow.compat.v1 as tf
x = tf.constant(5,tf.float32)
y = tf.constant([5], tf.float32)
z = tf.constant([5,3,4], tf.float32)
t = tf.constant([[5,3,4,6],[2,3,4,7]], tf.float32)
u = tf.constant([[[5,3,4,6],[2,3,4,0]]], tf.float32)
v = tf.constant([[[[5,3,4,6],[2,3,4,0]],
[[5,3,4,6],[2,3,4,0]],
[[5,3,4,6],[2,3,4,0]]],
], tf.float32)
print(v)
```

```
tf.Tensor(
[[[5. 3. 4. 6.]
 [2. 3. 4. 0.]]

 [[5. 3. 4. 6.]
 [2. 3. 4. 0.]]

 [[5. 3. 4. 6.]
 [2. 3. 4. 0.]]], shape=(3, 2, 4), dtype=float32)
```

[]:

→ `tf.constant()` is used to create an tensor object (cannot be changed during execution), has a fixed value and is not updated when training the model.

→ `tf.constant()` helps define constant tensors from the start. They are useful when you need sample data, fixed parameters, or values that do not change in the graph.

3.2.

3.2.1.

```
[8]: # Nguyễn Hoàng Tùng

import tensorflow.compat.v1 as tf
tf.compat.v1.disable_eager_execution()
x1 = tf.Variable(5.3, tf.float32)
x2 = tf.Variable(4.3, tf.float32)
x = tf.multiply(x1,x2)
init = tf.global_variables_initializer()
with tf.Session() as sess:
    sess.run(init)
    t = sess.run(x)
    print(t)
|
```

WARNING:tensorflow:From C:\Users\Admin\AppData\Local\Temp\ipykernel_16700\2810851454.py:4: The name tf.disable_eager_execution is deprecated. Please use tf.compat.v1.disable_eager_execution instead.

WARNING:tensorflow:From C:\Users\Admin\AppData\Local\Temp\ipykernel_16700\2810851454.py:8: The name tf.global_variables_initializer is deprecated. Please use tf.compat.v1.global_variables_initializer instead.

22.79

[]:

3.2.2.

```
[9]: # Nguyễn Hoàng Tùng

import tensorflow.compat.v1 as tf
tf.compat.v1.disable_eager_execution()
x1 = tf.Variable([[5.3,4.5,6.0],
[4.3,4.3,7.0]
], tf.float32)
x2 = tf.Variable([[4.3,4.3,7.0],
[5.3,4.5,6.0]
], tf.float32)
x = tf.multiply(x1,x2)
init = tf.global_variables_initializer()
with tf.Session() as sess:
    sess.run(init)
    t = sess.run(x)
    print(t)

[[22.79 19.35 42. ]
 [22.79 19.35 42.  ]]
```

[]:

3.2.3.

```
[10]: # Nguyễn Hoàng Tùng

import tensorflow.compat.v1 as tf
# creating nodes in computation graph
node = tf.Variable(tf.zeros([2,2]))
# running computation graph
with tf.Session() as sess:
    # initialize all global variables
    sess.run(tf.global_variables_initializer())
    # evaluating node
    print("Tensor value before addition:\n",sess.run(node))
    # elementwise addition to tensor
    node = node.assign(node + tf.ones([2,2]))
    # evaluate node again
    print("Tensor value after addition:\n", sess.run(node))
    sess.close()

Tensor value before addition:
[[0. 0.]
 [0. 0.]]
Tensor value after addition:
[[1. 1.]
 [1. 1.]]
```

[]:

→ `tf.Variable` is a tensor that can change its value during program execution, is often used to store model parameters (e.g. weights, bias) during neural network training.

3.3.

3.3.1.

[11]: # Nguyễn Hoàng Tùng

```
import tensorflow.compat.v1 as tf
tf.compat.v1.disable_eager_execution()
x = tf.placeholder(tf.float32, None)
y = tf.add(x, x)
with tf.Session() as sess:
    x_data = 5
    result = sess.run(y, feed_dict={x: x_data})
    print(result)
```

10.0

[]:

3.3.2.

[12]: # Nguyễn Hoàng Tùng

```
import tensorflow.compat.v1 as tf
tf.compat.v1.disable_eager_execution()
x = tf.placeholder(tf.float32, [None, 3])
y = tf.add(x, x)
with tf.Session() as sess:
    x_data = [[1.5, 2.0, 3.3]]
    result = sess.run(y, feed_dict={x: x_data})
    print(result)
```

[[3. 4. 6.6]]

[]:

3.3.3.

[13]: # Nguyễn Hoàng Tùng

```
import tensorflow.compat.v1 as tf
tf.compat.v1.disable_eager_execution()
x = tf.placeholder(tf.float32, [None, None, 3])
y = tf.add(x, x)
with tf.Session() as sess:
    x_data = [[[1, 2, 3]]]
    result = sess.run(y, feed_dict={x: x_data})
    print(result)
```

[[[2. 4. 6.]]]

[]:

3.3.4.

[15]: # Nguyễn Hoàng Tùng

```
import tensorflow.compat.v1 as tf
tf.compat.v1.disable_eager_execution()
x = tf.placeholder(tf.float32,[None,4,3])
y = tf.add(x,x)
with tf.Session() as sess:
    x_data= [[[1,2,3],
               [2,3,4],
               [2,3,5],
               [0,1,2]
              ]]
    result = sess.run(y,feed_dict={x:x_data})
    print(result)
```

```
[[[ 2.  4.  6.]
  [ 4.  6.  8.]
  [ 4.  6. 10.]
  [ 0.  2.  4.]]]
```

[]:

3.3.5.

[16]: import tensorflow.compat.v1 as tf

```
tf.compat.v1.disable_eager_execution()
x = tf.placeholder(tf.float32,[2,4,3])
y = tf.add(x,x)
with tf.Session() as sess:
    x_data= [[[1,2,3],
               [2,3,4],
               [2,3,5],
               [0,1,2]
              ],
              [[1,2,3],
               [2,3,4],
               [2,3,5],
               [0,1,2]
              ]]
    result = sess.run(y,feed_dict={x:x_data})
    print(result)
```

```
[[[ 2.  4.  6.]
  [ 4.  6.  8.]
  [ 4.  6. 10.]
  [ 0.  2.  4.]]]
```

```
[[[ 2.  4.  6.]
  [ 4.  6.  8.]
  [ 4.  6. 10.]
  [ 0.  2.  4.]]]
```

[]:

3.3.6.

```
[17]: # Nguyễn Hoàng Tùng

import tensorflow.compat.v1 as tf
tf.compat.v1.disable_eager_execution()
x = tf.placeholder(tf.float32,[2,4,3])
y = tf.placeholder(tf.float32,[2,4,3])
z = tf.add(x,y)
u = tf.multiply(x,y)
with tf.Session() as sess:
    x_data= [[1,2,3],
              [2,3,4],
              [2,3,5],
              [0,1,2]
              ],
              [[1,2,3],
              [2,3,4],
              [2,3,5],
              [0,1,2]
              ]]
    y_data= [[1,2,3],
              [2,3,4],
              [2,3,5],
              [0,1,2]
              ],
              [[1,2,3],
              [2,3,4],
              [2,3,5],
              [0,1,2]
              ]]
    result1 = sess.run(z,feed_dict={x:x_data, y:y_data})
    result2 = sess.run(u,feed_dict={x:x_data, y:y_data})
    print("result1 =", result1)
    print("result2 =", result2)

result1 = [[[ 2.  4.  6.]
 [ 4.  6.  8.]
 [ 4.  6. 10.]
 [ 0.  2.  4.]]

[[ 2.  4.  6.]
 [ 4.  6.  8.]
 [ 4.  6. 10.]
 [ 0.  2.  4.]]]
result2 = [[[ 1.  4.  9.]
 [ 4.  9. 16.]
 [ 4.  9. 25.]
 [ 0.  1.  4.]]

[[ 1.  4.  9.]
 [ 4.  9. 16.]
 [ 4.  9. 25.]
 [ 0.  1.  4.]]]
```

→ `tf.placeholder(dtype, shape)` is a placeholder for loading external data into the computation graph. The value of the placeholder will be provided when running the session using the `Feed_dict` parameter, often used to feed data trainers (inputs, labels) into the model. Placeholders do not contain available values, expected to be fed data at the time of the new run.

3.4.

3.4.1.

[18]: # Nguyễn Hoàng Tùng

```
import tensorflow.compat.v1 as tf
tf.compat.v1.disable_eager_execution()
x1 = tf.constant(5.3, tf.float32)
x2 = tf.constant(1.5, tf.float32)
w1 = tf.Variable(0.7, tf.float32)
w2 = tf.Variable(0.5, tf.float32)
u = tf.multiply(x1, w1)
v = tf.multiply(x2, w2)
z = tf.add(u, v)
result = tf.sigmoid(z)
init = tf.global_variables_initializer()
with tf.Session() as sess:
    sess.run(init)
    print(sess.run(result))
```

0.9885698

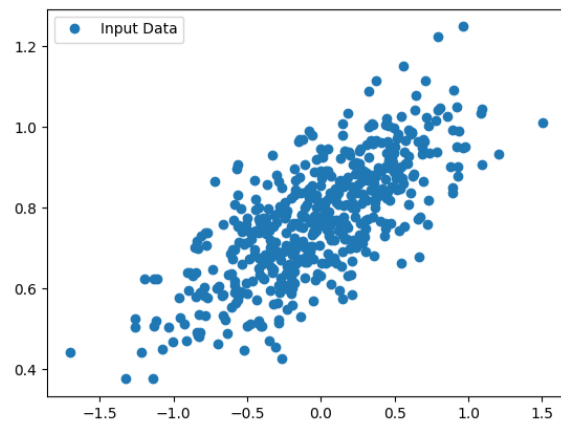
[]:

3.4.2.

[20]: # Nguyễn Hoàng Tùng

```
import numpy as np
import matplotlib.pyplot as plt
number_of_points = 500
x_point = []
y_point = []
a = 0.22
b = 0.78
for i in range(number_of_points):
    x = np.random.normal(0.0, 0.5)
    y = a*x + b + np.random.normal(0.0, 0.1)
    x_point.append([x])
    y_point.append([y])

plt.plot(x_point, y_point, 'o', label = 'Input Data')
plt.legend()
plt.show()
```



3.4.3.

```
[21]: import tensorflow.compat.v1 as tf
tf.compat.v1.disable_eager_execution()
x1 = tf.placeholder(tf.float32, [None, 3])
x2 = tf.placeholder(tf.float32, [None, 3])
w1 = tf.Variable([0.5, 0.4, 0.7], tf.float32)
w2 = tf.Variable([0.8, 0.5, 0.6], tf.float32)
u1 = tf.multiply(w1, x1)
u2 = tf.multiply(w2, x2)
v = tf.add(u1, u2)
z = tf.sigmoid(v)
init = tf.global_variables_initializer()
with tf.Session() as sess:
    x1_data = [[1, 2, 3]]
    x2_data = [[1, 2, 3]]
    sess.run(init)
    result = sess.run(z, feed_dict={x1: x1_data, x2: x2_data})
    print(result)
```

```
[[0.785835 0.85814893 0.9801597 ]]
```

```
[ ]:
```

3.4.4.

```
[22]: import tensorflow as tf
import numpy as np
matrix1 = np.array([(2, 2, 2), (2, 2, 2), (2, 2, 2)], dtype = 'int32')
matrix2 = np.array([(1, 1, 1), (1, 1, 1), (1, 1, 1)], dtype = 'int32')
print (matrix1)
print (matrix2)
matrix1 = tf.constant(matrix1)
matrix2 = tf.constant(matrix2)
matrix_product = tf.matmul(matrix1, matrix2)
matrix_sum = tf.add(matrix1, matrix2)
matrix_3 = np.array([(2, 7, 2), (1, 4, 2), (9, 0, 2)], dtype = 'float32')
print (matrix_3)
matrix_det = tf.matrix_determinant(matrix_3)
with tf.Session() as sess:
    result1 = sess.run(matrix_product)
    result2 = sess.run(matrix_sum)
    result3 = sess.run(matrix_det)
print (result1)
print (result2)
print (result3)
```

```
[[2 2 2]
 [2 2 2]
 [2 2 2]]
[[1 1 1]
 [1 1 1]
 [1 1 1]]
[[2. 7. 2.]
 [1. 4. 2.]
 [9. 0. 2.]]
```

→ An operation (op) in TensorFlow is a node in a computational graph that represents a calculation (addition, multiplication, sigmoid, matrix, ...). Constants, variables, and placeholders are data, and operations are actions that manipulate that data. To run an operation, you must call `sess.run(op)` or `.eval()` (in TF1.x).

3.5.

[27]: # Nguyễn Hoàng Tùng

```
# importing the dependencies
import tensorflow.compat.v1 as tf
import numpy as np
import matplotlib.pyplot as plt

# Model Parameters
learning_rate = 0.01
training_epochs = 2000
display_step = 200

# Training Data
train_X = np.asarray([3.3,4.4,5.5,6.71,6.93,4.168,9.779,6.182,7.59,2.167,7.042,10.791,5.313,7.997,5.654,9.27,3.1])
train_y = np.asarray([1.7,2.76,2.09,3.19,1.694,1.573,3.366,2.596,2.53,1.221,2.827,3.465,1.65,2.904,2.42,2.94,1.3])
n_samples = train_X.shape[0]

# Test Data
test_X = np.asarray([6.83, 4.668, 8.9, 7.91, 5.7, 8.7, 3.1, 2.1])
# Set placeholders for feature and target vectors
X = tf.placeholder(tf.float32)
y = tf.placeholder(tf.float32)

# Set model weights and bias
test_y = np.asarray([1.84, 2.273, 3.2, 2.831, 2.92, 3.24, 1.35, 1.03])
W = tf.Variable(np.random.randn(), name="weight")
b = tf.Variable(np.random.randn(), name="bias")

# Construct a linear model
linear_model = W*X + b

# Mean squared error
cost = tf.reduce_sum(tf.square(linear_model - y)) / (2*n_samples)

# Gradient descent
optimizer = tf.train.GradientDescentOptimizer(learning_rate).minimize(cost)

# Initializing the variables
init = tf.global_variables_initializer()

# Launch the graph
with tf.Session() as sess:
    # Load initialized variables in current session
    sess.run(init)
    # Fit all training data
    for epoch in range(training_epochs):
        # perform gradient descent step
        sess.run(optimizer, feed_dict={X: train_X, y: train_y})
```

```
# Display logs per epoch step
if (epoch+1) % display_step == 0:
    c = sess.run(cost, feed_dict={X: train_X, y: train_y})
    print("Epoch:{0:6} \t Cost:{1:10.4} \t W:{2:6.4} \t b:{3:6.4}".
          format(epoch+1, c, sess.run(W), sess.run(b)))

# Print final parameter values
print("Optimization Finished!")
training_cost = sess.run(cost, feed_dict={X: train_X, y: train_y})
print("Final training cost:", training_cost, "W:", sess.run(W), "b:", sess.run(b), '\n')

# Graphic display
plt.plot(train_X, train_y, 'ro', label='Original data')
plt.plot(train_X, sess.run(W) * train_X + sess.run(b), label='Fitted line')
plt.legend()
plt.show()

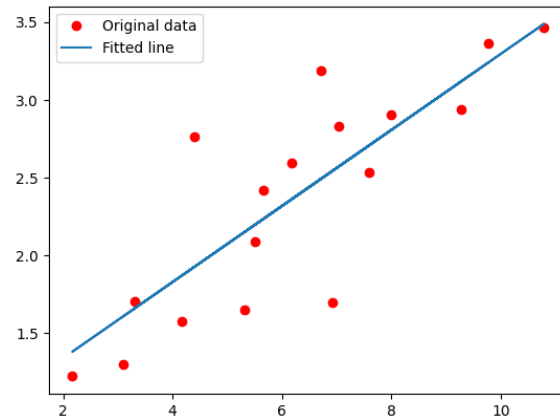
# Testing the model
testing_cost = sess.run(tf.reduce_sum(tf.square(linear_model - y)) / (2 * test_X.shape[0]),
                        feed_dict={X: test_X, y: test_y})

print("Final testing cost:", testing_cost)
print("Absolute mean square loss difference:", abs(training_cost - testing_cost))

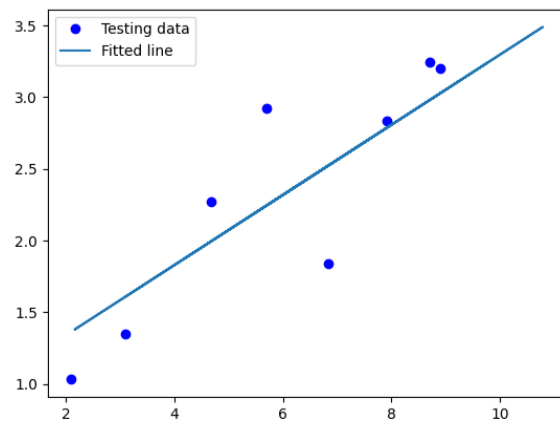
# Display fitted line on test data
plt.plot(test_X, test_y, 'bo', label='Testing data')
plt.plot(train_X, sess.run(W) * train_X + sess.run(b), label='Fitted line')
plt.legend()
plt.show()
```

Epoch: 200	Cost: 0.08955	W:0.1879	b: 1.25
Epoch: 400	Cost: 0.08469	W:0.2017	b: 1.153
Epoch: 600	Cost: 0.0817	W:0.2125	b: 1.077
Epoch: 800	Cost: 0.07987	W:0.2209	b: 1.017
Epoch: 1000	Cost: 0.07874	W:0.2275	b:0.9697
Epoch: 1200	Cost: 0.07804	W:0.2327	b:0.9328
Epoch: 1400	Cost: 0.07761	W:0.2368	b:0.9039
Epoch: 1600	Cost: 0.07735	W: 0.24	b:0.8813
Epoch: 1800	Cost: 0.07719	W:0.2425	b:0.8635
Epoch: 2000	Cost: 0.07709	W:0.2445	b:0.8495

Optimization Finished!
Final training cost: 0.07708812 W: 0.24448015 b: 0.8495255



Final testing cost: 0.07991749
Absolute mean square loss difference: 0.0028293729



→ Explanation

Dependencies

```
1. import tensorflow.compat.v1 as tf
2. import numpy as np
3. import matplotlib.pyplot as plt
4. tf.compat.v1.disable_eager_execution()
5.
```

- Using **TensorFlow 1.x style** (Session, placeholder).
- numpy → for data handling.
- matplotlib → for plotting.

Hyperparameters

```
1. learning_rate = 0.01
2. training_epochs = 2000
3. display_step = 200
4.
```

- **Learning rate:** how big the gradient descent update steps are.
- **Epochs:** how many iterations we train.
- **Display step:** how often to print training progress.

Training & Test Data

```
1. train_X = np.asarray([...])
2. train_y = np.asarray([...])
3. n_samples = train_X.shape[0]
4.
5. test_X = np.asarray([...])
6. test_y = np.asarray([...])
7.
```

- Training data → used to learn the parameters W , b .
- Test data → used later to evaluate performance.

Placeholders (inputs/outputs)

```
1. X = tf.placeholder(tf.float32)
2. y = tf.placeholder(tf.float32)
3.
```

- X and y are **placeholders** → we will feed actual values during training using `feed_dict`.

Model Parameters

```
1. W = tf.Variable(np.random.randn(), name="weight")
2. b = tf.Variable(np.random.randn(), name="bias")
3.
```

- Trainable **parameters** that TensorFlow will optimize.
- Initialized randomly.

Model Definition

```
1. linear_model = W*X + b
2.
```

- Predicted value ($\hat{y}$) is a straight line.

Cost Function (Mean Squared Error)

```
1. cost = tf.reduce_sum(tf.square(linear_model - y)) / (2*n_samples)
2.
```

- Measures how far predictions are from actual values.
- Divided by 2*n_samples (classical linear regression formula).

Optimizer

```
1. optimizer = tf.train.GradientDescentOptimizer(learning_rate).minimize(cost)
2.
```

- Gradient Descent updates W and b to minimize the cost function.

Training Process

```
1. init = tf.global_variables_initializer()
2. with tf.Session() as sess:
3.     sess.run(init)
4.     for epoch in range(training_epochs):
5.         sess.run(optimizer, feed_dict={X: train_X, y: train_y})
6.
```

- Initialize variables.
- Run training loop → optimizer adjusts W and b each step.

Logging Training Progress

```
1. if (epoch+1) % display_step == 0:
2.     c = sess.run(cost, feed_dict={X: train_X, y: train_y})
3.     print("Epoch:", epoch+1, "Cost:", c, "W:", sess.run(W), "b:", sess.run(b))
4.
```

- Every 200 epochs → print current cost, weight, and bias.
- Helps track learning progress.

Final Results & Visualization

```
1. print("Final training cost:", training_cost, "W:", sess.run(W), "b:", sess.run(b))
2.
3. plt.plot(train_X, train_y, 'ro', label='Original data')
```

```
4. plt.plot(train_X, sess.run(W) * train_X + sess.run(b), label='Fitted line')
5. plt.legend()
6. plt.show()
7.
```

- After training, display learned line vs training data.
- Red dots = original data, line = model's prediction.

Testing the Model

```
1. testing_cost = sess.run(
2.     tf.reduce_sum(tf.square(linear_model - y)) / (2 * test_X.shape[0]),
3.     feed_dict={X: test_X, y: test_y})
4.
```

- Compute test cost (MSE) to see generalization.
- Compare training vs testing performance.

3.6.

[31]: # Nguyễn Hoàng Tùng



```
import numpy as np
import tensorflow.compat.v1 as tf
tf.disable_eager_execution() # make sure we are in TF1.x graph mode

# -----
# NumPy Implementation
# -----
def neuron_numpy(X, W, b, activation="tanh"):
    # Weighted sum
    z = np.dot(X, W) + b

    # Apply activation
    if activation == "tanh":
        return np.tanh(z)
    elif activation == "sigmoid":
        return 1 / (1 + np.exp(-z))
    elif activation == "relu":
        return np.maximum(0, z)
    else:
        return z # linear (no activation)

# -----
# TensorFlow v1 Implementation
# -----
def neuron_tf(X, W, b, activation="tanh"):
    # Define constants
    X_tf = tf.constant(X, dtype=tf.float32)
    W_tf = tf.constant(W, dtype=tf.float32)
    b_tf = tf.constant(b, dtype=tf.float32)

    # Weighted sum
    z = tf.reduce_sum(X_tf * W_tf) + b_tf

    # Choose activation
    if activation == "tanh":
        act = tf.math.tanh(z)
    elif activation == "sigmoid":
        act = tf.math.sigmoid(z)
    elif activation == "relu":
        act = tf.nn.relu(z)
    else:
        act = z # linear

    # Run inside a session
    with tf.Session() as sess:
        return sess.run(act)

# -----
# Example usage
# -----
X = [2.0, 3.0, 4.0]
W = [0.05, 0.07, 0.09]
b = 0.60

print("=== NumPy Implementation ===")
print("tanh:", neuron_numpy(X, W, b, "tanh"))
print("sigmoid:", neuron_numpy(X, W, b, "sigmoid"))
print("relu:", neuron_numpy(X, W, b, "relu"))
print("linear:", neuron_numpy(X, W, b, None))

print("\n=== TensorFlow v1 Implementation ===")
print("tanh:", neuron_tf(X, W, b, "tanh"))
print("sigmoid:", neuron_tf(X, W, b, "sigmoid"))
print("relu:", neuron_tf(X, W, b, "relu"))
print("linear:", neuron_tf(X, W, b, None))
```

```
=== NumPy Implementation ===
tanh: 0.8537976531552436
sigmoid: 0.7807427479121283
relu: 1.27
linear: 1.27

=== TensorFlow v1 Implementation ===
tanh: 0.8537976
sigmoid: 0.78074276
relu: 1.27
linear: 1.27
```

[]:

→ Explanation

- **Weighted sum** → Each input is multiplied by its corresponding weight, then all are summed with a bias.
- **Activation function (tanh)** → Introduces **non-linearity** into the model.
 - Without it, the neuron would just compute a linear function.
 - With tanh, the output is squeezed into the range $[-1, 1]$.
- **Output** → The final transformed value (here ≈ 0.8538) is what flows to the next layer or becomes the prediction.

3.7.

```
[32]: # Nguyễn Hoàng Tùng

import tensorflow.compat.v1 as tf
import numpy as np
tf.disable_eager_execution() # Use TF1.x style

# -----
# Define network parameters
# -----
n_inputs = 3      # Age, Hours of study, Previous scores
n_hidden1 = 5     # neurons in first hidden layer
n_hidden2 = 5     # neurons in second hidden layer
n_outputs = 2     # Pass / Fail

learning_rate = 0.01
training_epochs = 1000

# -----
# Placeholders for input/output
# -----
X = tf.placeholder(tf.float32, [None, n_inputs]) # input features
y = tf.placeholder(tf.float32, [None, n_outputs]) # target labels (one-hot)

# -----
# Weights & Biases
# -----
weights = {
    'h1': tf.Variable(tf.random_normal([n_inputs, n_hidden1])),
    'h2': tf.Variable(tf.random_normal([n_hidden1, n_hidden2])),
    'out': tf.Variable(tf.random_normal([n_hidden2, n_outputs]))
}
biases = {
    'b1': tf.Variable(tf.zeros([n_hidden1])),
    'b2': tf.Variable(tf.zeros([n_hidden2])),
    'out': tf.Variable(tf.zeros([n_outputs]))
}

# -----
# Build Model
# -----
# Layer 1
layer1 = tf.add(tf.matmul(X, weights['h1']), biases['b1'])
layer1 = tf.nn.relu(layer1)
```

```

# Layer 2
layer2 = tf.add(tf.matmul(layer1, weights['h2']), biases['b2'])
layer2 = tf.nn.relu(layer2)

# Output Layer
logits = tf.add(tf.matmul(layer2, weights['out']), biases['out'])
output = tf.nn.softmax(logits) # probabilities of pass/fail

# -----
# Define Loss & Optimizer
# -----
loss = tf.reduce_mean(tf.nn.softmax_cross_entropy_with_logits_v2(labels=y, logits=logits))
optimizer = tf.train.AdamOptimizer(learning_rate=learning_rate).minimize(loss)

# -----
# Define Accuracy
# -----
correct_pred = tf.equal(tf.argmax(output, 1), tf.argmax(y, 1))
accuracy = tf.reduce_mean(tf.cast(correct_pred, tf.float32))

# -----
# Example Dummy Data
# -----
# Features: [Age, Hours of Study, Previous Score]
train_X = np.array([[18, 5, 70],
                    [19, 8, 85],
                    [17, 2, 40],
                    [20, 10, 90]], dtype=np.float32)

# Labels: Pass = [1,0], Fail = [0,1]
train_y = np.array([[1,0],
                    [1,0],
                    [0,1],
                    [1,0]], dtype=np.float32)

```

```

# -----
# Training
# -----
init = tf.global_variables_initializer()

with tf.Session() as sess:
    sess.run(init)

    for epoch in range(training_epochs):
        _, c = sess.run([optimizer, loss], feed_dict={X: train_X, y: train_y})

        if (epoch+1) % 200 == 0:
            acc = sess.run(accuracy, feed_dict={X: train_X, y: train_y})
            print("Epoch:", epoch+1, "Loss=", c, "Accuracy=", acc)

    print("\nOptimization Finished!")

# Final prediction
pred = sess.run(output, feed_dict={X: train_X})
print("Predicted Probabilities:\n", pred)

```

```

Epoch: 200 Loss= 0.56233513 Accuracy= 0.75
Epoch: 400 Loss= 0.56233513 Accuracy= 0.75
Epoch: 600 Loss= 0.56233513 Accuracy= 0.75
Epoch: 800 Loss= 0.5623352 Accuracy= 0.75
Epoch: 1000 Loss= 0.5623352 Accuracy= 0.75

```

```

Optimization Finished!
Predicted Probabilities:
[[0.75000006 0.24999999]
 [0.75000006 0.24999999]
 [0.75000006 0.24999999]
 [0.75000006 0.24999999]]

```

[]:

→ Explanation

1. Input placeholders:

X stores input data (3 features).

y stores expected output (Pass/Fail, one-hot encoded).

2. Weights & Biases:

Randomly initialized for each layer. These are trainable.

3. **Model Layers:**

- $\text{layer1} = \text{ReLU}(XW + b)$
- $\text{layer2} = \text{ReLU}(\text{layer1}W + b)$
- $\text{output} = \text{softmax}(\text{logits})$

4. **Loss:**

`softmax_cross_entropy` measures how well predictions match labels.

5. **Optimizer:**

`AdamOptimizer` adjusts weights using gradient descent.

6. **Accuracy:**

Compares predicted class vs true class.

7. **Training Loop:**

Runs 1000 epochs. Every 200 steps, prints loss and accuracy.

8. **Prediction:**

At the end, prints probability of **Pass** vs **Fail** for each input.

3.8.

1. Explanation of "model, layer, batch, epoch, structure, loss, optimization, gradient" with simple examples:

- **Model** A deep learning **model** is fundamentally a **graph of layers** that maps input data to predictions. In Keras, models are typically instances of the `Model` class, which can be simple `Sequential` stacks of layers or more complex multi-input, multi-output networks.
 - **Example:** For classifying handwritten digits (like in the MNIST dataset), a model might consist of a sequence of two `Dense` layers, where the final layer is a 10-way `softmax` classification layer to output probabilities for each digit (0-9). You can visualize a deep network as a multi-stage information-distillation process, where data passes through successive filters, becoming increasingly purified and informative for a specific task.
- **Layer** A **layer** is the **central class of Keras** and serves as a fundamental building block of a neural network. Each layer encapsulates certain **weights** (learnable parameters) and performs specific **computations**.

- **Example:** A Dense layer (also called a fully connected layer) implements an affine transformation: $\text{output} = \text{relu}(\text{dot}(\text{input}, W) + b)$, where W and b are the layer's weights, and relu is an activation function. The number of "units" in a Dense layer determines the dimensionality of the representation space it learns. Other examples include Conv2D layers for image processing and LSTM or GRU layers for sequence data.
- **Batch** Deep learning models do not process an entire dataset at once. Instead, they divide the data into **small subsets called batches or mini-batches**. These batches are typically between 8 and 128 samples in size, often a power of 2 to optimize GPU memory allocation.
 - **Example:** In the MNIST handwritten digit classification, a batch of 128 images would be processed simultaneously. During training, one batch is used to compute a single gradient-descent update to the model's weights.
- **Epoch** An **epoch** refers to **one complete iteration over all samples in the training data**. During training, a model is typically exposed to the entire training dataset for a specified number of epochs.
 - **Example:** Training the movie review classification model for "20 epochs" means the model will iterate through the entire training dataset 20 times, with each iteration consisting of multiple mini-batch updates.
- **Structure** The **structure** of a neural network refers to its **architecture or topology**, which is essentially how layers are organized and connected. This structure defines the "hypothesis space," or the predefined space of possibilities, within which the machine learning algorithm searches for useful data representations.
 - **Example:** A Sequential model in Keras represents a simple stack of layers. More complex structures include networks with residual connections, multi-head attention (like in the Transformer architecture), or multiple input/output branches. The choice of structure (e.g., Dense network for tabular data, convnet for images, RNN for timeseries) is crucial as it encodes assumptions about the data's inherent structure.
- **Loss (function)** The **loss function** (also known as the objective function or cost function) is a critical component that **measures how well the network's output (predictions) aligns with the true target values**. It quantifies the "distance" or mismatch between what the model predicted and what it was supposed to predict.

- **Example:** For binary classification (e.g., classifying movie reviews as positive or negative), `binary_crossentropy` is a common loss function. For regression problems (e.g., predicting house prices), `mean_squared_error` (MSE) is widely used. The loss function's score acts as a feedback signal to guide the model's learning process. It must be differentiable for gradient descent to work.
- **Optimization** is the process by which a machine learning model **updates its internal parameters (weights) to improve its performance by minimizing the loss function**. This is typically achieved through gradient-based optimization algorithms.
 - **Example: Stochastic Gradient Descent (SGD)** is a fundamental optimization algorithm. Variants of SGD, known as **optimizers**, like RMSprop, Adam, or Adagrad, are used in practice. The optimizer dictates the exact way the gradient of the loss is used to adjust parameters.
- **Gradient** A **gradient** is essentially the **derivative of a tensor operation**. In the context of neural networks, it indicates the direction of the steepest increase of the loss function with respect to the model's weights.
 - **Example:** If you want to decrease the loss, you move the model's weights in the **opposite direction of the gradient**. The **Backpropagation algorithm** is the central mechanism in deep learning for efficiently computing these gradients across all layers of a neural network. Modern frameworks like TensorFlow provide tools like GradientTape for automatic differentiation, so practitioners don't have to manually compute gradients.

2. Explanation of Figures 2.21, 2.22, 2.23, 2.24, 2.25:

These figures visually explain the concept of **computation graphs** and the **Backpropagation algorithm**, which is fundamental to how neural networks learn.

- **Figure 2.21: The computation graph representation of our two-layer model** This figure depicts a **directed acyclic graph (DAG)**, which is the underlying data structure in TensorFlow for representing computations. It illustrates a two-layer neural network model, showing how various inputs (e.g., `x` for input data, `W1`, `b1`, `W2`, `b2` for weights, and `y_true` for true targets) flow through a series of **tensor operations** (e.g., dot product, relu activation, + addition, softmax activation, and loss function) to produce a final `loss_val`. This graph allows for the treatment of computation as data, which is crucial for features like automatic differentiation and distributed computing.

- Figure 2.22: A basic example of a computation graph** This is a simplified version of a computation graph, used to make the explanation of backpropagation clearer by using scalar variables instead of tensors. It shows inputs (x , b , w , y_{true}) passing through elementary operations ($*$, $+$, loss) to produce intermediate results (x_1 , x_2) and ultimately a loss_val . The goal in this context is to compute the gradients of loss_val with respect to the learnable parameters w and b to minimize the loss.
- Figure 2.23: Running a forward pass** This figure demonstrates the "**forward pass**" on the simplified computation graph from Figure 2.22. It shows numerical values being propagated through the graph from the inputs to the output. For example, if $x=2$ and $w=3$, then x_1 becomes $2*3 = 6$. If $x_1=6$ and $b=1$, then x_2 becomes $6+1 = 7$. Finally, if $y_{\text{true}}=4$ and $x_2=7$, the loss_val is $\text{abs}(4-7) = 3$. The forward pass is the process of computing the model's predictions and loss for a given input.
- Figure 2.24: Running a backward pass** This figure introduces the concept of the "**backward pass**" as part of backpropagation. It shows the computation graph from Figure 2.23 "reversed," with each edge annotated by a local derivative, $\text{grad}(B, A)$, which represents how much B varies when A varies. For instance, $\text{grad}(\text{loss_val}, x_2) = 1$ indicates that the change in loss_val is directly proportional to the change in x_2 in this specific example. This process begins at the loss_val and works backward through the graph to compute derivatives for all intermediate nodes and input parameters.
- Figure 2.25: Path from loss_val to w in the backward graph** This figure specifically illustrates how the **chain rule** is applied within the backward pass. It highlights the path from loss_val back to the parameter w . According to the chain rule, the gradient of loss_val with respect to w ($\text{grad}(\text{loss_val}, w)$) is found by multiplying the derivatives along this path: $\text{grad}(\text{loss_val}, x_2) * \text{grad}(x_2, x_1) * \text{grad}(x_1, w)$. In this example, $1 * 1 * 2$ results in $\text{grad}(\text{loss_val}, w) = 2$. This multiplication of local derivatives is the core of backpropagation, allowing efficient computation of how each parameter contributes to the final loss.

3. Understanding of section 3.6: "Anatomy of a neural network: Understanding core Keras APIs"

Section 3.6 serves as a foundational guide to understanding the structure and operational aspects of neural networks within the Keras framework, a high-level API for TensorFlow. My understanding of this section is as follows:

The section establishes that a deep learning model is essentially a **graph of interconnected layers**, with the Model class (and its subclass Sequential) being central to Keras for building

these structures. While Sequential models are straightforward stacks of layers, the more flexible Functional API (and Model subclassing) allows for complex network **structures** like multi-input/output systems or residual connections. The choice of network topology is crucial as it defines the "hypothesis space"—the set of possible functions the model can learn.

A key part of the workflow is the **compile() step**, where the learning process is configured before training. This involves selecting three vital components:

1. **Loss function:** This is the objective that the model will minimize during training, acting as a measure of success for the given task. For example, `binary_crossentropy` for binary classification, `categorical_crossentropy` for multiclass classification, and `mean_squared_error` for regression. The importance of choosing a loss function that accurately reflects the problem's goal is heavily emphasized, warning that a poorly chosen one can lead to unintended "shortcuts" by the model.
2. **Optimizer:** This dictates how the model's **weights** are updated based on the **gradient** of the loss function, implementing a variant of stochastic gradient descent. `rmsprop` is presented as a generally good default choice.
3. **Metrics:** These are measures used to monitor the model's performance during training and validation, such as "accuracy". Unlike the loss function, metrics do not need to be differentiable as they are not directly optimized.

The section then underscores the critical role of **validation data**. It explains that a model's performance must be monitored on data it has never seen during training (the validation set) to assess its **generalization** ability and to detect **overfitting**, where the model performs well on training data but poorly on new data. This monitoring is done by passing `validation_data` to the `fit()` method.

Finally, the section briefly covers **inference**, which is the process of using a trained model (via methods like `predict()`) to generate outputs for new, unseen inputs in a practical setting.

In summary, section 3.6 provides a comprehensive "anatomical" and "instructional" view of how to construct, configure, and evaluate Keras-based neural networks, covering the roles of layers, models, loss functions, optimizers, and the crucial practice of validating performance for generalization. It equips the reader with the core Keras APIs necessary for practical deep learning applications.

3.9.

1. Processing MNIST with Keras

Explanation of the Keras Approach:

The Keras workflow typically involves four main steps:

1. **Loading and Preparing Data:** Keras preloads common datasets like MNIST, which can then be easily reshaped and normalized.
2. **Defining the Model Architecture:** A model is built as a graph of layers, with Sequential models being simple stacks of layers. Each layer (Dense, Conv2D, etc.) encapsulates weights and performs specific computations.
3. **Compiling the Model:** This step configures the learning process by specifying the optimizer (how weights are updated), loss function (what to minimize), and metrics (what to monitor).
4. **Training and Evaluating the Model:** The `fit()` method runs the training loop, processing data in batches over several epochs, and `evaluate()` assesses performance on new data.

Keras Code for MNIST Classification:

```
[2]: # Nguyễn Hoàng Tùng

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.datasets import mnist
import numpy as np

# Ensure eager mode is enabled for Keras
tf.compat.v1.enable_eager_execution()

# 1. Load and prepare the MNIST dataset
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()

train_images = train_images.reshape((60000, 28 * 28))
train_images = train_images.astype("float32") / 255

test_images = test_images.reshape((10000, 28 * 28))
test_images = test_images.astype("float32") / 255

# 2. Define the model architecture
model = keras.Sequential([
    layers.Dense(512, activation="relu"),
    layers.Dense(10, activation="softmax")
])

# 3. Compile the model
model.compile(
    optimizer="rmsprop",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

# 4. Train
print("Training Keras model...")
history = model.fit(train_images, train_labels, epochs=5, batch_size=128)

# 5. Evaluate
print("\nEvaluating Keras model on test data...")
test_loss, test_acc = model.evaluate(test_images, test_labels)
print(f"Keras Test Accuracy: {test_acc:.4f}")

# Predictions
print("\nGenerating predictions with Keras model...")
predictions = model.predict(test_images)
print(f"First test image predicted class: {np.argmax(predictions[0])}")
print(f"First test image true label: {test_labels[0]}")
|
```

```

Training Keras model...
Epoch 1/5
469/469 ————— 4s 7ms/step - accuracy: 0.9230 - loss: 0.2656
Epoch 2/5
469/469 ————— 3s 6ms/step - accuracy: 0.9685 - loss: 0.1079
Epoch 3/5
469/469 ————— 5s 6ms/step - accuracy: 0.9789 - loss: 0.0702
Epoch 4/5
469/469 ————— 3s 6ms/step - accuracy: 0.9844 - loss: 0.0509
Epoch 5/5
469/469 ————— 3s 7ms/step - accuracy: 0.9886 - loss: 0.0380

Evaluating Keras model on test data...
313/313 ————— 1s 2ms/step - accuracy: 0.9814 - loss: 0.0590
Keras Test Accuracy: 0.9814

Generating predictions with Keras model...
313/313 ————— 1s 2ms/step
First test image predicted class: 7
First test image true label: 7

```

Explanation of the Keras Code:

- **`mnist.load_data()`**: Downloads and loads the MNIST dataset, returning training and test images/labels as NumPy arrays.
- **`reshape()` and `astype("float32") / 255`**: Transforms each 28x28 image into a 784-element vector and normalizes pixel values from 0 to 1 for better network performance.
- **`keras.Sequential([...])`**: Creates a model by stacking layers sequentially.
- **`layers.Dense(units, activation="relu")`**: Adds a densely connected layer. The relu activation introduces non-linearity, allowing the model to learn complex transformations.
- **`layers.Dense(10, activation="softmax")`**: The final layer outputs 10 probabilities, one for each digit class. softmax ensures these probabilities sum to 1.
- **`model.compile(...)`**: Configures the training process. `rmsprop` is a common optimizer, `sparse_categorical_crossentropy` is a suitable loss for integer labels in multiclass classification, and `accuracy` is a metric to track.
- **`model.fit(...)`**: Trains the model. `epochs` specifies how many times the entire training dataset is passed through the network, and `batch_size` determines the number of samples per gradient update.
- **`model.evaluate(...)`**: Calculates the loss and metrics (e.g., accuracy) on the test data to assess generalization performance.
- **`model.predict(...)`**: Generates probability predictions for new input data.

2. Processing MNIST WITHOUT Keras (Pure TensorFlow)

Explanation of the Pure TensorFlow Approach:

Building a neural network without Keras's high-level Sequential and fit() involves:

1. **Manual Layer and Model Definition:** Instead of `keras.layers.Dense`, you define a custom Dense layer class that holds `tf.Variable` instances for weights and biases, and implements the forward pass. A Sequential model class is also defined to chain these custom layers.
2. **Data Handling:** A `BatchGenerator` is implemented to manually iterate over the dataset in mini-batches.
3. **Loss and Optimizer:** Loss functions (like `sparse_categorical_crossentropy`) and optimizers (like `RMSprop`) are used directly from `tf.keras.losses` and `tf.keras.optimizers`, as they are core TensorFlow components.
4. **Custom Training Loop:** This is the core difference. Instead of `model.fit()`, a manual loop iterates through epochs and batches. For each batch:
 - **Forward Pass:** Input data flows through the custom layers to produce predictions.
 - **Loss Computation:** The predictions are compared to true labels using a chosen loss function.
 - **Backward Pass (Gradient Calculation):** `tf.GradientTape` records operations to automatically compute gradients of the loss with respect to all `tf.Variables` (weights and biases).
 - **Weight Update:** The optimizer uses these gradients to update the model's weights in the direction that minimizes the loss.
5. **Evaluation Loop:** Similar to training, a manual loop processes test data in batches to calculate overall loss and accuracy.

Pure TensorFlow Code for MNIST Classification:

[3]: # Nguyễn Hoàng Tùng

```
import tensorflow as tf
import numpy as np
from tensorflow.keras.datasets import mnist # Using Keras for dataset loading as it's part of TensorFlow.

# 0. Set up eager execution explicitly for GradientTape (default in TF2.x)
# tf.compat.v1.disable_eager_execution() is often used for graph mode, but we want eager for tape.
# In TF 2.x, eager execution is default, so no explicit call is usually needed.

# 1. Manual Layer and Model Definition
# Implements the logic of a Keras Dense Layer manually.
class NaiveDense:
    def __init__(self, input_size, output_size, activation=None):
        self.activation = activation if activation is not None else lambda x: x
        # Initialize weights with small random values.
        w_shape = (input_size, output_size)
        w_initial_value = tf.random.uniform(w_shape, minval=0., maxval=1e-1)
        self.W = tf.Variable(w_initial_value)
        # Initialize biases with zeros.
        b_shape = (output_size,)
        b_initial_value = tf.zeros(b_shape)
        self.b = tf.Variable(b_initial_value)

    def __call__(self, inputs):
        # Forward pass: output = activation(dot(input, W) + b).
        return self.activation(tf.matmul(inputs, self.W) + self.b)

    @property
    def trainable_weights(self):
        # Returns the Layer's learnable parameters.
        return [self.W, self.b]

# Manual Sequential model to stack NaiveDense Layers.
class NaiveSequential:
    def __init__(self, layers):
        self.layers = layers

    def __call__(self, inputs):
        x = inputs
        for layer in self.layers:
            x = layer(x)
        return x

    @property
    def trainable_weights(self):
        weights = []
        for layer in self.layers:
            weights.extend(layer.trainable_weights)
        return weights
```

```

# 2. Data Handling with a BatchGenerator
class BatchGenerator:
    def __init__(self, images, labels, batch_size=128):
        assert len(images) == len(labels)
        self.index = 0
        self.images = images
        self.labels = labels
        self.batch_size = batch_size
        self.num_batches = len(images) // batch_size

    def next(self):
        images = self.images[self.index : self.index + self.batch_size]
        labels = self.labels[self.index : self.index + self.batch_size]
        self.index += self.batch_size
        return images, labels

# Load and prepare the MNIST dataset
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()

train_images = train_images.reshape((60000, 28 * 28)).astype("float32") / 255
test_images = test_images.reshape((10000, 28 * 28)).astype("float32") / 255

# Define the custom model
model = NaiveSequential([
    NaiveDense(input_size=28 * 28, output_size=512, activation=tf.nn.relu),
    NaiveDense(input_size=512, output_size=10, activation=tf.nn.softmax)
])

# 3. Define Loss Function and Optimizer
# Using Keras's built-in sparse_categorical_crossentropy and RMSprop, as they are part of TensorFlow's core.
loss_fn = tf.keras.losses.SparseCategoricalCrossentropy()
optimizer = tf.keras.optimizers.RMSprop(learning_rate=1e-3)
accuracy_metric = tf.keras.metrics.SparseCategoricalAccuracy()

# 4. Custom Training Step
@tf.function # Decorator to compile the function into a TensorFlow graph for speed.
def one_training_step_tf_pure(model, images_batch, labels_batch, loss_fn, optimizer, accuracy_metric):
    with tf.GradientTape() as tape:
        predictions = model(images_batch)
        loss = loss_fn(labels_batch, predictions) # Compute Loss.

    # Compute gradients of the loss with respect to the model's trainable weights.
    gradients = tape.gradient(loss, model.trainable_weights)

    # Update weights using the optimizer and computed gradients.
    optimizer.apply_gradients(zip(gradients, model.trainable_weights))

    # Update accuracy metric.
    accuracy_metric.update_state(labels_batch, predictions)
    return loss

```

```

# 5. Full Custom Training Loop
print("\nTraining pure TensorFlow model...")
num_epochs = 5
for epoch in range(num_epochs):
    print(f"\nEpoch {epoch + 1}/{num_epochs}")
    batch_generator = BatchGenerator(train_images, train_labels) # Re-initialize for each epoch
    accuracy_metric.reset_state() # Reset metric at the start of each epoch.

    for batch_counter in range(batch_generator.num_batches):
        images_batch, labels_batch = batch_generator.next()
        loss = one_training_step_tf_pure(model, images_batch, labels_batch, loss_fn, optimizer, accuracy_metric)

        if batch_counter % 100 == 0:
            print(f"  Batch {batch_counter}/{batch_generator.num_batches}: Loss={loss.numpy():.4f}, Accuracy={accuracy_metric.result().numpy():.4f}")

    print(f"End of Epoch {epoch + 1}: Training Accuracy={accuracy_metric.result().numpy():.4f}")

# 6. Custom Evaluation Loop
print("\nEvaluating pure TensorFlow model on test data...")
test_batch_generator = BatchGenerator(test_images, test_labels)
test_accuracy_metric = tf.keras.metrics.SparseCategoricalAccuracy()
test_loss_sum = 0.0
total_test_samples = 0

for batch_counter in range(test_batch_generator.num_batches):
    images_batch, labels_batch = test_batch_generator.next()
    predictions = model(images_batch)
    loss = loss_fn(labels_batch, predictions)
    test_loss_sum += loss.numpy() * len(images_batch)
    total_test_samples += len(images_batch)
    test_accuracy_metric.update_state(labels_batch, predictions)

final_test_loss = test_loss_sum / total_test_samples
final_test_accuracy = test_accuracy_metric.result().numpy()
print(f"Pure TensorFlow Test Loss: {final_test_loss:.4f}, Pure TensorFlow Test Accuracy: {final_test_accuracy:.4f}")

# Generate predictions with pure TensorFlow model
print("\nGenerating predictions with pure TensorFlow model...")
# For predictions, we can just call the model with test_images
test_predictions = model(test_images)
predicted_class = np.argmax(test_predictions.numpy())
print(f"First test image predicted class: {predicted_class}")
print(f"First test image true label: {test_labels}")

```

```

Training pure TensorFlow model...

Epoch 1/5
  Batch 0/468: Loss=4.7807, Accuracy=0.1016
  Batch 100/468: Loss=0.5077, Accuracy=0.6148
  Batch 200/468: Loss=0.5109, Accuracy=0.7029
  Batch 300/468: Loss=0.3473, Accuracy=0.7470
  Batch 400/468: Loss=0.5286, Accuracy=0.7729
End of Epoch 1: Training Accuracy=0.7889

Epoch 2/5
  Batch 0/468: Loss=0.2312, Accuracy=0.9453
  Batch 100/468: Loss=0.4314, Accuracy=0.8897
  Batch 200/468: Loss=0.2065, Accuracy=0.8895
  Batch 300/468: Loss=0.3157, Accuracy=0.8897
  Batch 400/468: Loss=0.4510, Accuracy=0.8912
End of Epoch 2: Training Accuracy=0.8946

Epoch 3/5
  Batch 0/468: Loss=0.1883, Accuracy=0.9531
  Batch 100/468: Loss=0.2047, Accuracy=0.9117
  Batch 200/468: Loss=0.1974, Accuracy=0.9122
  Batch 300/468: Loss=0.2618, Accuracy=0.9129
  Batch 400/468: Loss=0.3309, Accuracy=0.9145
End of Epoch 3: Training Accuracy=0.9173

Epoch 4/5
  Batch 0/468: Loss=0.1517, Accuracy=0.9609
  Batch 100/468: Loss=0.1264, Accuracy=0.9360
  Batch 200/468: Loss=0.1531, Accuracy=0.9373
  Batch 300/468: Loss=0.1643, Accuracy=0.9386
  Batch 400/468: Loss=0.2139, Accuracy=0.9400
End of Epoch 4: Training Accuracy=0.9424

Epoch 5/5
  Batch 0/468: Loss=0.1090, Accuracy=0.9609
  Batch 100/468: Loss=0.0930, Accuracy=0.9578
  Batch 200/468: Loss=0.0939, Accuracy=0.9586
  Batch 300/468: Loss=0.1204, Accuracy=0.9599
  Batch 400/468: Loss=0.1595, Accuracy=0.9601
End of Epoch 5: Training Accuracy=0.9619

Evaluating pure TensorFlow model on test data...
Pure TensorFlow Test Loss: 0.1348, Pure TensorFlow Test Accuracy: 0.9592

Generating predictions with pure TensorFlow model...
First test image predicted class: 4400
First test image true label: [7 2 1 ... 4 5 6]

```

Explanation of the Pure TensorFlow Code:

- **NaiveDense and NaiveSequential classes:** These replace `tf.keras.layers.Dense` and `tf.keras.Sequential`. They manually define `tf.Variable` objects for weights (W) and biases (b), and implement the forward pass logic. The `trainable_weights` property is crucial for `tf.GradientTape` to identify parameters to optimize.
- **BatchGenerator class:** This utility manually handles splitting the dataset into mini-batches for iteration during training and evaluation.
- **tf.keras.losses.SparseCategoricalCrossentropy()** and **tf.keras.optimizers.RMSprop():** Although they have "keras" in their path, these are fundamental TensorFlow components for loss calculation and gradient-based optimization.
- **tf.keras.metrics.SparseCategoricalAccuracy():** Used to track accuracy, resetting its state at the beginning of each epoch or evaluation cycle.
- **one_training_step_tf_pure function:** This function encapsulates the core training logic for a single batch.

- **with tf.GradientTape() as tape::** This context manager records all operations performed within it to compute gradients later.
 - **predictions = model(images_batch):** Performs the forward pass using our custom NaiveSequential model.
 - **loss = loss_fn(labels_batch, predictions):** Calculates the loss between the model's predictions and the true labels.
 - **gradients = tape.gradient(loss, model.trainable_weights):** The GradientTape computes the gradient of the loss with respect to all tf.Variables in model.trainable_weights.
 - **optimizer.apply_gradients(zip(gradients, model.trainable_weights)):** Applies the calculated gradients to update the model's weights, moving them in the direction that reduces the loss.
- **@tf.function:** This decorator compiles the Python function into a static TensorFlow graph, significantly speeding up execution, especially for repetitive operations in training loops.
 - **Manual Training Loop:** Iterates through epochs and, for each epoch, creates a BatchGenerator to process data in batches. It calls one_training_step_tf_pure for each batch and prints progress.
 - **Manual Evaluation Loop:** Similar to the training loop, it iterates through test data batches, calculates loss and updates accuracy metric without applying gradients. This gives an unbiased measure of the model's generalization performance.
 - **model(test_images) for prediction:** After training, the model can be directly called with new inputs to generate predictions.

3.10.

1. Key Concepts from Chapter 4:

- **Machine Learning Tasks:**
 - **Binary Classification:** Categorizing input samples into two exclusive classes, such as classifying movie reviews as positive or negative.

- **Multiclass Classification:** Categorizing input samples into more than two exclusive classes, like classifying newswires into multiple topics or handwritten digits (like MNIST).
- **Scalar Regression:** Predicting a continuous scalar value, such as estimating house prices. Vector regression for multiple continuous values is also mentioned.
- **Data Preprocessing:** Neural networks require data to be formatted into appropriately preprocessed floating-point tensors.
 - **Vectorization:** Converting raw data (like text) into numerical tensors. For text, this often involves multi-hot encoding (representing words as binary vectors) or using integer sequences with embedding layers.
 - **Normalization:** Scaling features in input data, especially when they have different ranges, to ensure they all take small values on a similar scale (e.g., dividing by 255 for image pixels, or standardizing features to have mean 0 and standard deviation 1 for numerical data).
- **Model Architecture Principles:**
 - **Dense Layers:** Often used as the core building blocks, implementing $\text{output} = \text{relu}(\text{dot}(\text{input}, W) + b)$. The number of units (dimensionality of representation space) in these layers is a key architectural decision.
 - **Output Layer and Activation Functions:** The choice depends on the task.
 - **Binary Classification:** A single Dense unit with sigmoid activation, outputting a probability between 0 and 1.
 - **Multiclass Classification (single-label):** A Dense layer with units equal to the number of classes and a softmax activation, outputting a probability distribution over the classes.
 - **Scalar/Vector Regression:** A Dense layer with units equal to the number of values to predict and **no activation function**.
- **Loss Functions:** The quantity minimized during training, representing how well the model's predictions match the targets.
 - **Binary Classification:** `binary_crossentropy`.
 - **Multiclass Classification:** `categorical_crossentropy` (for one-hot encoded labels) or `sparse_categorical_crossentropy` (for integer labels).

- **Regression:** mean_squared_error (MSE) or mean_absolute_error (MAE).
- **Optimizers:** Determines how the network's weights are updated based on the loss function, typically a variant of stochastic gradient descent (SGD). rmsprop is generally a good default choice.
- **Model Evaluation:**
 - **Validation Sets:** A subset of data held apart from training to monitor model accuracy and loss on new, unseen data during training, crucial for detecting overfitting.
 - **Overfitting:** The phenomenon where a model performs worse on new data than on its training data, indicating it has memorized training patterns rather than generalizing.
 - **K-fold Cross-Validation:** A technique used when little data is available to reliably evaluate a model, by splitting data into K partitions and training K models.
- **Common-Sense Baselines:** Essential for determining if a machine learning model is making real progress, by comparing its performance to a trivial non-machine learning approach.

Chapter 4 emphasizes the **end-to-end machine learning workflow**, from data preprocessing and model architecture design to evaluation, and the crucial importance of monitoring performance on data outside the training set to prevent overfitting.

2. Running code with datasets MNIST and IMDB

[14]: # Nguyễn Hoàng Tùng

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import numpy as np
import matplotlib.pyplot as plt # For optional plotting

print("--- MNIST Dataset (Multiclass Classification) ---")

# 1. Load the MNIST dataset
# MNIST is a classic dataset for handwritten digit classification (multiclass)
(train_images_mnist, train_labels_mnist), (test_images_mnist, test_labels_mnist) = keras.datasets.mnist.load_data()

# 2. Prepare the data
# Reshape images to 28*28 (784 features) and normalize pixel values to
train_images_mnist = train_images_mnist.reshape((60000, 28 * 28)).astype("float32") / 255
test_images_mnist = test_images_mnist.reshape((10000, 28 * 28)).astype("float32") / 255

# 3. Build the model
# A simple densely connected network is used for multiclass classification.
# The last layer has 10 units (for 10 classes) and a softmax activation for probability distribution.
model_mnist = keras.Sequential([
    layers.Dense(512, activation="relu"),
    layers.Dense(10, activation="softmax")
])

# 4. Compile the model
# Using 'rmsprop' optimizer and 'sparse_categorical_crossentropy' because labels are integers.
model_mnist.compile(optimizer="rmsprop",
                    loss="sparse_categorical_crossentropy",
                    metrics=["accuracy"])

# 5. Train the model
print("Training MNIST model...")
# Training for a few epochs, showing progress.
model_mnist.fit(train_images_mnist, train_labels_mnist, epochs=5, batch_size=128)

# 6. Evaluate the model
print("Evaluating MNIST model...")
test_loss_mnist, test_acc_mnist = model_mnist.evaluate(test_images_mnist, test_labels_mnist)
print(f"MNIST Test Accuracy: {test_acc_mnist:.3f}")
```

```
print("\n--- IMDB Dataset (Binary Classification) ---")

# 1. Load the IMDB dataset
# IMDB is a dataset of movie reviews for binary (positive/negative) classification.
# num_words=10000 keeps only the top 10,000 most frequent words.
(train_data_imdb, train_labels_imdb), (test_data_imdb, test_labels_imdb) = keras.datasets.imdb.load_data(num_words=10000)

# 2. Prepare the data
# Convert integer sequences to multi-hot encoded binary vectors (10,000 dimensions).
def vectorize_sequences(sequences, dimension=10000):
    results = np.zeros((len(sequences), dimension))
    for i, sequence in enumerate(sequences):
        for j in sequence:
            results[i, j] = 1.
    return results

x_train_imdb = vectorize_sequences(train_data_imdb)
x_test_imdb = vectorize_sequences(test_data_imdb)

# Vectorize labels as float32
y_train_imdb = np.asarray(train_labels_imdb).astype("float32")
y_test_imdb = np.asarray(test_labels_imdb).astype("float32")

# 3. Building your model
# For binary classification, the last layer has 1 unit and 'sigmoid' activation.
model_imdb = keras.Sequential([
    layers.Dense(16, activation="relu"),
    layers.Dense(16, activation="relu"),
    layers.Dense(1, activation="sigmoid")
])

# 4. Compile the model
# Using 'rmsprop' optimizer and 'binary_crossentropy' loss.
model_imdb.compile(optimizer="rmsprop",
                  loss="binary_crossentropy",
                  metrics=["accuracy"])

# 5. Set aside a validation set
# 10,000 samples from the training data are used for validation.
x_val_imdb = x_train_imdb[:10000]
partial_x_train_imdb = x_train_imdb[10000:]
y_val_imdb = y_train_imdb[:10000]
partial_y_train_imdb = y_train_imdb[10000:]
```



```

# 6. Train the model
print("Training IMDB model...")
# The validation_data argument is used to monitor performance on unseen data.
history_imdb = model_imdb.fit(partial_x_train_imdb,
                              partial_y_train_imdb,
                              epochs=20, # Training for 20 epochs as in the textbook example
                              batch_size=512,
                              validation_data=(x_val_imdb, y_val_imdb))

# 7. Evaluate the model
print("Evaluating IMDB model...")
# The textbook retrains for 4 epochs on full data after validation. For this example, we evaluate directly.
results_imdb = model_imdb.evaluate(x_test_imdb, y_test_imdb)
loss_imdb, acc_imdb = results_imdb
print(f"IMDB Test Loss: {loss_imdb:.3f}")
print(f"IMDB Test Accuracy: {acc_imdb:.3f}")

# 8. Generate predictions
print("Generating predictions for IMDB test set (first 5 samples):")
predictions_imdb = model_imdb.predict(x_test_imdb[:5])
print(predictions_imdb)

```

```

--- MNIST Dataset (Multiclass Classification) ---
Training MNIST model...
Epoch 1/5
469/469 ————— 45s 7ms/step - accuracy: 0.9251 - loss: 0.2634
Epoch 2/5
469/469 ————— 5s 7ms/step - accuracy: 0.9688 - loss: 0.1059
Epoch 3/5
469/469 ————— 4s 8ms/step - accuracy: 0.9789 - loss: 0.0700
Epoch 4/5
469/469 ————— 3s 7ms/step - accuracy: 0.9844 - loss: 0.0511
Epoch 5/5
469/469 ————— 5s 7ms/step - accuracy: 0.9890 - loss: 0.0376
Evaluating MNIST model...
313/313 ————— 1s 2ms/step - accuracy: 0.9799 - loss: 0.0619
MNIST Test Accuracy: 0.980

```

```

--- IMDB Dataset (Binary Classification) ---
Training IMDB model...
Epoch 1/20
30/30 ————— 2s 29ms/step - accuracy: 0.7777 - loss: 0.5209 - val_accuracy: 0.8392 - val_loss: 0.4144
Epoch 2/20
30/30 ————— 0s 12ms/step - accuracy: 0.8955 - loss: 0.3207 - val_accuracy: 0.8863 - val_loss: 0.3091
Epoch 3/20
30/30 ————— 1s 16ms/step - accuracy: 0.9186 - loss: 0.2422 - val_accuracy: 0.8885 - val_loss: 0.2827
Epoch 4/20
30/30 ————— 1s 14ms/step - accuracy: 0.9342 - loss: 0.1970 - val_accuracy: 0.8868 - val_loss: 0.2768
Epoch 5/20
30/30 ————— 1s 16ms/step - accuracy: 0.9443 - loss: 0.1653 - val_accuracy: 0.8878 - val_loss: 0.2770
Epoch 6/20
30/30 ————— 1s 16ms/step - accuracy: 0.9551 - loss: 0.1395 - val_accuracy: 0.8781 - val_loss: 0.3056
Epoch 7/20
30/30 ————— 1s 11ms/step - accuracy: 0.9607 - loss: 0.1231 - val_accuracy: 0.8808 - val_loss: 0.2987
Epoch 8/20
30/30 ————— 0s 11ms/step - accuracy: 0.9689 - loss: 0.1036 - val_accuracy: 0.8810 - val_loss: 0.3072
Epoch 9/20
30/30 ————— 1s 12ms/step - accuracy: 0.9729 - loss: 0.0916 - val_accuracy: 0.8732 - val_loss: 0.3563
Epoch 10/20
30/30 ————— 1s 13ms/step - accuracy: 0.9805 - loss: 0.0762 - val_accuracy: 0.8799 - val_loss: 0.3477
Epoch 11/20
30/30 ————— 1s 16ms/step - accuracy: 0.9823 - loss: 0.0667 - val_accuracy: 0.8751 - val_loss: 0.3599
Epoch 12/20
30/30 ————— 1s 11ms/step - accuracy: 0.9862 - loss: 0.0568 - val_accuracy: 0.8728 - val_loss: 0.4065
Epoch 13/20
30/30 ————— 0s 15ms/step - accuracy: 0.9883 - loss: 0.0490 - val_accuracy: 0.8764 - val_loss: 0.3946
Epoch 14/20
30/30 ————— 0s 14ms/step - accuracy: 0.9918 - loss: 0.0399 - val_accuracy: 0.8657 - val_loss: 0.4423
Epoch 15/20
30/30 ————— 1s 15ms/step - accuracy: 0.9921 - loss: 0.0378 - val_accuracy: 0.8664 - val_loss: 0.4909
Epoch 16/20
30/30 ————— 1s 15ms/step - accuracy: 0.9954 - loss: 0.0289 - val_accuracy: 0.8666 - val_loss: 0.5093
Epoch 17/20
30/30 ————— 1s 14ms/step - accuracy: 0.9975 - loss: 0.0232 - val_accuracy: 0.8730 - val_loss: 0.4922
Epoch 18/20
30/30 ————— 0s 14ms/step - accuracy: 0.9967 - loss: 0.0232 - val_accuracy: 0.8721 - val_loss: 0.5070
Epoch 19/20
30/30 ————— 0s 14ms/step - accuracy: 0.9983 - loss: 0.0160 - val_accuracy: 0.8649 - val_loss: 0.5817
Epoch 20/20
30/30 ————— 1s 15ms/step - accuracy: 0.9956 - loss: 0.0198 - val_accuracy: 0.8689 - val_loss: 0.5507
Evaluating IMDB model...
782/782 ————— 2s 2ms/step - accuracy: 0.8581 - loss: 0.5999
IMDB Test Loss: 0.600
IMDB Test Accuracy: 0.858
Generating predictions for IMDB test set (first 5 samples):
1/1 ————— 0s 63ms/step
[[0.01599958]
 [0.99999666]
 [0.2458226 ]
 [0.93224436]
 [0.99387485]]

```

Explanation of the Code:

MNIST Dataset (Multiclass Classification):

1. **Loading Data:** The `mnist.load_data()` function directly loads the dataset, which consists of 60,000 training images and 10,000 test images of handwritten digits (0-9). This is a **multiclass classification** problem.
2. **Data Preparation:** The images, initially 28x28 pixel arrays, are reshaped into 784-dimensional vectors. Pixel values (0-255) are converted to float32 and normalized to the range by dividing by 255. This is a standard **data preprocessing** step for numerical stability.
3. **Model Building:** A `keras.Sequential` model is built with two Dense layers. The first layer has 512 units and uses a `relu` activation function. The final Dense layer has 10 units (one for each digit class) and a `softmax` activation, which outputs a probability distribution over the 10 classes.
4. **Model Compilation:** The model is configured for training using the `compile()` method. The `rmsprop` **optimizer** is chosen, which is a generally effective choice. The loss function is `sparse_categorical_crossentropy`, appropriate for multiclass classification when labels are integers. `accuracy` is specified as a **metric** to monitor performance.
5. **Model Training:** The `fit()` method trains the model on the `train_images_mnist` and `train_labels_mnist` for 5 epochs. Each epoch iterates over the data in batches of 128 samples.
6. **Model Evaluation:** The `evaluate()` method calculates the loss and accuracy on the `test_images_mnist` and `test_labels_mnist`, providing a final measure of the model's performance on unseen data.

IMDB Dataset (Binary Classification):

1. **Loading Data:** The `imdb.load_data()` function loads 50,000 movie reviews, preprocessed into sequences of integers, where each integer represents a word. `num_words=10000` limits the vocabulary to the 10,000 most frequent words. This is a **binary classification** problem (positive/negative sentiment).
2. **Data Preparation:** The integer sequences representing reviews are converted into **multi-hot encoded** binary vectors of 10,000 dimensions. Each vector is mostly zeros, with ones at the indices corresponding to words present in the review. Labels (0s and 1s) are also cast to float32.

3. **Model Building:** A `keras.Sequential` model is constructed with three Dense layers. The first two intermediate layers use `relu` activation with 16 units. The final Dense layer has 1 unit and a `sigmoid` activation, which outputs a probability of the review being positive (0-1), suitable for **binary classification**.
4. **Model Compilation:** The model uses the `rmsprop` optimizer and `binary_crossentropy` as the **loss function**, which is standard for binary classification problems where the output is a probability. `accuracy` is again used as a **metric**.
5. **Validation Set:** A subset of 10,000 samples from the `x_train_imdb` and `y_train_imdb` is set aside as a **validation set** (`x_val_imdb`, `y_val_imdb`). This is a crucial step to monitor the model's performance on data it hasn't seen during training, helping to identify and prevent **overfitting**.
6. **Model Training:** The `fit()` method trains the model on the `partial_x_train_imdb` and `partial_y_train_imdb`. The `validation_data` argument points to the validation set, allowing Keras to compute and display validation loss and accuracy at the end of each epoch. This allows practitioners to observe the model's generalization performance during training.
7. **Model Evaluation:** After training, the `evaluate()` method assesses the model's performance on the completely unseen `x_test_imdb` and `y_test_imdb`.
8. **Generating Predictions:** Finally, the `predict()` method is used to generate likelihood scores (probabilities) for the first 5 samples of the test set, demonstrating how the trained model can be used for **inference** on new data.